

Verificación y Validación del Sistema RegistraMe: Informe Técnico Integral del Proceso de Pruebas de Software

Denise Andrea Huacani Jara
Universidad Nacional de San Agustín
Arequipa, Perú
dhuacani@unsa.edu.pe

Fabiana Francinet Pacheco Palo
Universidad Nacional de San Agustín
Arequipa, Perú
fpachecop@unsa.edu.pe

Fernando Miguel Garambel Marín
Universidad Nacional de San Agustín
Arequipa, Perú
fgarambel@unsa.edu.pe

Jeans Anthony Ajra Huacso
Universidad Nacional de San Agustín
Arequipa, Perú
jajra@unsa.edu.pe

Luis Guillermo Luque Condori
Universidad Nacional de San Agustín
Arequipa, Perú
lluquecon@unsa.edu.pe

Ryan Fabian Valdivia Segovia
Universidad Nacional de San Agustín
Arequipa, Perú
rvaldiviase@unsa.edu.pe

Sergio Danilo Hanco Mullisaca
Universidad Nacional de San Agustín
Arequipa, Perú
shanccom@unsa.edu.pe

Resumen—RegistraMe es un sistema de gestión integral para ópticas, compuesto por un backend en Django REST Framework y un frontend en React, orientado a digitalizar la operación de un centro óptico: ventas, caja, inventario, clientes y recetas clínicas. Este informe documenta el proceso completo de verificación y validación aplicado al proyecto a lo largo de sus cuatro sprints, estructurado en cinco niveles de prueba: pruebas unitarias de caja blanca sobre el backend (518 pruebas automatizadas con `pytest`, 87 % de cobertura documentada), pruebas funcionales manuales de caja negra sobre los módulos críticos, pruebas de integración backend (10 escenarios), pruebas de sistema de extremo a extremo con Playwright (17 escenarios de negocio materializados en 72 ejecuciones automatizadas sobre tres motores de navegador), pruebas no funcionales de rendimiento y seguridad con Locust y `pytest` (17 escenarios) y una sesión formal de pruebas de aceptación (UAT) con un usuario externo real (12 criterios ejecutados). La estrategia de selección de casos se fundamentó en las técnicas de Partición de Equivalencia y Análisis de Valores Límite de Myers, el Principio de la Mochila de Spillner para las pruebas no funcionales, y el modelo de calidad ISO/IEC 25010, bajo la estructura documental de ISO/IEC/IEEE 29119-3. Los resultados muestran que el sistema protege correctamente sus recursos (7/7 escenarios de seguridad aprobados) y preserva la integridad transaccional bajo concurrencia crítica y ante fallos de infraestructura, pero no satisface de forma generalizada los criterios de rendimiento bajo carga (5 de 8 escenarios de estrés y volumen no aprobados), y la sesión de aceptación con el usuario final concluyó en un veredicto de *aprobación condicionada* (91.6 % de criterios validados), identificando defectos concretos de validación de datos que quedan documentados para el backlog. Se presenta también la matriz de trazabilidad módulo-nivel de prueba y una discusión de los hallazgos que orienta el trabajo de optimización previo a un despliegue de producción con alta concurrencia.

Index Terms—pruebas de software, verificación y validación, pruebas de extremo a extremo, pruebas de rendimiento, pruebas de aceptación, ISO/IEC 25010, IEEE 29119, RegistraMe

I. INTRODUCCIÓN

RegistraMe es un sistema de gestión integral desarrollado para un centro óptico, cuyo propósito es automatizar la emisión de órdenes de venta, el control de caja, la gestión de inventario, el registro de clientes y la administración de recetas ópticas de un local. El proyecto fue desarrollado por el equipo “La Machaca” a lo largo de cuatro sprints, entre el 28 de mayo y el 19 de julio de 2026, dentro del curso de Pruebas de Software de la Escuela Profesional de Ingeniería de Sistemas de la Universidad Nacional de San Agustín.

A diferencia de un ejercicio de pruebas aislado sobre un módulo específico, este proyecto exigió construir un proceso de aseguramiento de calidad completo sobre un sistema real de dos capas -backend en Django REST Framework y frontend en React- con reglas de negocio no triviales: control de stock en tiempo real, cuadre de caja, estados de venta con transiciones (PENDIENTE → PARCIAL → PAGADO → ANULADO), control de acceso por rol y consultas puntuales de DNI y RUC contra una API pública de terceros (`apis.net.pe`). Verificar un sistema de este tamaño obliga a decidir, con criterio explícito y no arbitrario, qué probar, con qué técnica y en qué nivel; probar todo exhaustivamente es inviable, y omitir pruebas sin justificación deja huecos de calidad indetectables hasta producción.

Este informe documenta el proceso de verificación y validación aplicado íntegramente al proyecto: la estrategia y las técnicas de diseño de casos empleadas, los cinco niveles de prueba ejecutados a lo largo de los cuatro sprints, los resultados obtenidos en cada uno -incluidos los defectos encontrados- y la discusión de los hallazgos que se desprenden del proceso

completo. El objetivo no es reportar un único tipo de prueba, sino consolidar en un solo documento técnico, con el rigor de un artículo de conferencia, la totalidad del trabajo de calidad realizado sobre RegistraMe: desde la prueba unitaria de un método de modelo hasta la validación de negocio de un usuario externo real.

El resto del documento se organiza como sigue. La Sección II describe la arquitectura y el objeto bajo prueba. La Sección III presenta el marco normativo, las técnicas de diseño de casos y la organización del equipo. Las Secciones IV–IX documentan, en orden de nivel de prueba, los resultados de las pruebas unitarias, funcionales manuales, de integración, de sistema E2E, no funcionales y de aceptación, respectivamente. La Sección X presenta la matriz de trazabilidad y cobertura de calidad. La Sección XI describe la integración continua. La Sección XII discute los hallazgos principales y la Sección XIII concluye el informe.

II. OBJETO BAJO PRUEBA: ARQUITECTURA DE REGISTRAME

II-A. Arquitectura general

RegistraMe se organiza en tres capas. El *frontend* es una aplicación de página única construida con React 19, Vite y TailwindCSS, empaquetable como aplicación de escritorio mediante Tauri. Expone ocho páginas principales: Login, Dashboard, Punto de Venta (/sale-point, con subrutas de apertura y cierre de caja), Ventas (/sales), Inventario (/inventory), Prescripciones (/prescriptions), Reportes (/reports) y Configuración (/settings). El *backend* es una API REST construida con Django 5.2 y Django REST Framework, organizada en diez módulos de dominio: `users`, `products`, `sales`, `cash`, `clients`, `categories`, `suppliers`, `sequences`, `opticalCenter` y `external_services`. La persistencia se realiza sobre PostgreSQL 15. El frontend se comunica con el backend mediante HTTP/JSON autenticado con JSON Web Tokens (JWT). El módulo `external_services` implementa dos endpoints de proxy, `consultar_dni` y `consultar_ruc`, que reenvían la solicitud a la API pública de terceros `apis.net.pe`. De los dos, únicamente la consulta de DNI está conectada a un flujo de usuario real: el servicio `reniecService.ts` del frontend la invoca desde el panel de registro de clientes para autocompletar el nombre a partir del documento, y esa integración fue validada en vivo con un DNI real durante la sesión de UAT (Sección IX). El endpoint `consultar_ruc` existe y está cubierto por pruebas unitarias y de integración a nivel de API, pero no tiene ningún consumidor en el frontend: el campo de RUC del formulario de proveedores se completa manualmente y solo se valida su formato (11 dígitos), sin consulta real a la API externa en ningún flujo de la aplicación.

II-B. Roles y niveles de acceso

El sistema implementa control de acceso basado en roles (RBAC) con cinco niveles funcionales, resumidos en la Ta-

bla I. Los roles Cajero y Vendedor comparten el nivel 2 de acceso y conforman, a efectos de pruebas, una única clase de equivalencia.

Tabla I
ROLES Y NIVELES DE ACCESO DEL SISTEMA.

Rol	Nivel	Alcance
Gerente	0	Acceso total a todas las rutas
Supervisor	1	Acceso total, salvo POS directo
Cajero / Vendedor	2	Dashboard, Punto de Venta, Ventas
Logística	3	Dashboard, Inventario
Óptometra	4	Dashboard, Prescripciones

II-C. Requerimientos no funcionales relevantes

El documento de requerimientos del proyecto establece, entre otros, que el sistema debe responder a las operaciones de registro, consulta y venta en un máximo de 3 segundos (RNF-01), garantizar comunicación cifrada mediante HTTPS (RNF-03), y mantener una disponibilidad mínima del 99 % mensual (RNF-04). Estos requerimientos, junto con el modelo de calidad ISO/IEC 25010, son el criterio de referencia frente al cual se evalúan los resultados de las pruebas no funcionales reportadas en la Sección VIII.

III. METODOLOGÍA Y ESTRATEGIA DE PRUEBAS

III-A. Marco normativo

El proceso de pruebas se documentó siguiendo la estructura de ISO/IEC/IEEE 29119-3 para documentación de pruebas de software [1], empleada como “plan paraguas” bajo el cual se articulan los planes subordinados de cada nivel. La selección de casos de prueba funcionales se rige por las técnicas de Partición de Equivalencia (PE) y Análisis de Valores Límite (AVL) de Myers [2], formalizadas también en ISO/IEC/IEEE 29119-4 [3]. Las pruebas no funcionales se diseñaron bajo el Principio de la Mochila de Spillner [4] -tomar los flujos funcionales ya verificados y medir su comportamiento bajo condiciones extremas, en vez de diseñar flujos nuevos- y bajo las características de calidad de ISO/IEC 25010 [5]. La sesión de aceptación se diseñó conforme a la distinción de Pressman entre prueba Alfa y Beta [6], ejecutándose como una prueba Beta formal con un usuario externo real.

III-B. Técnicas de selección y descarte de casos

Probar cada combinación posible de entradas es computacionalmente inviable e innecesario: la Partición de Equivalencia divide el dominio de entrada en clases donde todo valor produce un comportamiento equivalente, y solo se prueba un representante de cada clase. Por ejemplo, el catálogo contiene del orden de 100 variantes de montura por material (Acetato, Metal, TR90, Titanio, Nylon); dado que el flujo de venta es idéntico para cualquiera de ellas, se seleccionó un único representante (la montura “PEGASUS”, de material Metal) para el escenario E2E-POS-02, y las 99 variantes restantes se descartaron por redundancia. De forma análoga, de los cinco roles del sistema se seleccionaron dos representantes

-gerentel para la clase de acceso total y vendedor1 para la clase de acceso operativo- ya que Cajero y Vendedor comparten nivel de acceso.

El Análisis de Valores Límite complementa a la PE probando exactamente en la frontera del dominio y justo fuera de ella, dado que los defectos de software se concentran desproporcionadamente en esos puntos [2]. Este criterio se aplicó de forma sistemática: stock del producto en el límite mínimo vendible (stock = 1) frente a una solicitud de una unidad adicional; monto de apertura de caja en S/ 0 frente a un monto válido; adelanto de pago en S/ 0 frente al total exacto de la venta; RUC de proveedor con longitud insuficiente; y, en el plano no funcional, concurrencia de 10 usuarios sobre un único recurso con stock = 1 para forzar una condición de carrera.

III-C. Niveles de prueba ejecutados

El proceso se estructuró en cinco niveles de prueba, ejecutados progresivamente a lo largo de los cuatro sprints del proyecto y resumidos en la Tabla II. Cada nivel se documenta en detalle en las Secciones IV-IX.

Tabla II
NIVELES DE PRUEBA EJECUTADOS SOBRE REGISTRAME.

Nivel	Cantidad	Herramienta	Sprint
Unitarias (caja blanca)	518 pruebas	pytest + coverage.py	2-4
Funcionales (caja negra, manual)	~60 escenarios	Ejecución manual QA	2
Integración (backend)	10 escenarios	pytest + PostgreSQL	3
Sistema E2E	17 escenarios / 72 ejec.	Playwright	4
No funcionales	17 escenarios	Locust + pytest	4
Aceptación (UAT)	12 escenarios	Sesión con usuario real	4

III-D. Organización del equipo

El equipo asumió roles complementarios de tipo ISTQB/IEEE durante el Sprint 4: un Test Lead responsable de la estrategia y de este artículo; un Test Environment Manager a cargo de la infraestructura y de la prueba de recuperabilidad; un UAT Facilitator responsable de las 15 sesiones de aceptación documentadas; dos Test Automation Engineers responsables de los 17 escenarios E2E de Playwright (POS/Caja e Inventario/Clientes, respectivamente); y dos Technical Test Analysts responsables de las pruebas de rendimiento y de las pruebas de seguridad y autenticación. Aunque la ejecución técnica se dividió por especialización para optimizar los tiempos del sprint, todos los integrantes mantuvieron conocimiento transversal de la arquitectura completa y de los resultados de todas las fases, condición verificada durante la sustentación del proyecto.

III-E. Entorno y herramientas

La Tabla III resume el entorno de ejecución. Los cinco niveles de prueba comparten los mismos datos semilla (database_seed.sql) y los mismos cinco usuarios de prueba, uno por nivel de acceso, lo que garantiza reproducibilidad entre niveles.

Tabla III
ENTORNO E INFRAESTRUCTURA DE PRUEBAS.

Componente	Detalle
Backend	Django 5.2 + PostgreSQL 15
Frontend	React 19 + Vite + TailwindCSS
Unitarias/Integración	pytest, pytest-django [7], coverage.py, DRF APIClient [8]
Sistema E2E	Playwright [9] (TypeScript) — Chromium, Firefox, WebKit
Rendimiento	Locust [10] 2.x (Python)
Seguridad	pytest + DRF APIClient
CI/CD	GitHub Actions — deploy-qa.yml, deploy-develop.yml
Datos semilla	database_seed.sql

IV. PRUEBAS UNITARIAS

IV-A. Alcance y diseño

Las pruebas unitarias cubren los diez módulos del backend mediante pytest y pytest-django, aislando cada unidad de las dependencias externas mediante unittest.mock: las 12 pruebas de external_services mockean por completo requests.get -ninguna llega a golpear la API real de apis.net.pe-, y de forma análoga se simulan la impresión térmica y el sistema de archivos, para que la suite se ejecute de forma determinista y sin red. Se verifican modelos (creación, validaciones, métodos __str__, propiedades calculadas, restricciones de base de datos), serializers de DRF (validación de entrada, transformación de salida) y ViewSets (autenticación, permisos, CRUD, filtros y acciones personalizadas), siguiendo un enfoque a posteriori: las pruebas se escribieron sobre código ya existente para asentar una base de regresión.

IV-B. Resultados documentados (Sprint 2) y estado actual

La última ejecución íntegramente documentada de la suite unitaria, realizada el 11 de junio de 2026, registró 475 pruebas distribuidas en 14 archivos, con 447 aprobadas y 28 fallidas (94.11 % de éxito) y una cobertura de código del 87 % (2935 de 3302 sentencias), medida con coverage.py en 3 minutos 35 segundos de ejecución total. La Tabla IV resume la distribución de pruebas por módulo en ese corte.

Como verificación de vigencia de este informe, se reconfirmó el 19 de julio de 2026 -fecha de cierre del proyecto- que la suite completa colecciona sin errores de importación ni de sintaxis: pytest-collect-only reporta 518 pruebas sobre los mismos 23 archivos de prueba del backend, un crecimiento del 9 % sobre el corte de Sprint 2 producto del endurecimiento continuo de la suite durante los Sprints 3 y 4.

Tabla IV
DISTRIBUCIÓN DE PRUEBAS UNITARIAS POR MÓDULO (CORTE
11/06/2026).

Módulo	Pruebas	Cobertura destacada
sales (incl. printer)	126	models.py 91 %, printer.py 99 %
products	65	models.py 99 %
cash	63	models.py, serializers.py 100 %
users	63	models.py, permissions.py 100 %
clients	43	filters.py, models.py 100 %
suppliers	46	models.py 100 %, serializers.py 80 %
categories	32	models.py 100 %
opticalCenter	14	serializers.py 62 % (mínimo del proyecto)
external_services	12	views.py 97 %
sequences	11	models.py 100 %
Total	475	87 % global

IV-C. Defectos identificados y corregidos

Los 28 fallos detectados el 11/06/2026 se agruparon y corrigieron por módulo. El caso más ilustrativo del valor de la prueba unitaria de caja blanca es el defecto que afectó a cinco pruebas de `TestPermissions` en el módulo `users`: la clase de permisos personalizada evaluaba la variable `role` en una expresión generadora que en realidad iteraba sobre `rol` (`role.nivel == X for rol in request.user.roles.all()`), produciendo un `NameError` que habría bloqueado en tiempo de ejecución cualquier verificación de nivel de acceso. Otros defectos representativos incluyeron un estado de venta que no transicionaba automáticamente de `PENDIENTE` a `PAGADO` tras registrar el pago completo, un `ViewSet` de Lunas con el método `POST` inhabilitado en el enrutador (HTTP 405 en lugar de 201), y un modelo `Supplier` que no invocaba `full_clean()` y por tanto no rechazaba correos con formato inválido. Todos los defectos de este corte quedaron registrados con causa raíz documentada, y su corrección se verificó en las ejecuciones subsiguientes que llevaron el conteo de 475 a 518 pruebas.

V. PRUEBAS FUNCIONALES MANUALES

V-A. Diseño y ejecución

Complementando la caja blanca del nivel unitario, se ejecutó una campaña de pruebas funcionales de caja negra sobre siete bloques del sistema: Gestión de Clientes, Punto de Venta/Caja, Gestión de Ventas, Logística e Inventario, Dashboard, Reportes y Estadísticas, y Control de Usuarios. El diseño combinó Partición de Equivalencia, Análisis de Valores Límite, validación de entradas mal formateadas, pruebas de transición de estados y tablas de decisión para reglas con múltiples condiciones (por ejemplo, permisos por rol combinados con visibilidad de información financiera). La ejecución se realizó de forma manual sobre un entorno desplegado, con evidencia fotográfica por caso y registro en hoja de cálculo colaborativa.

V-B. Resultados

De los casos ejecutados sobre Clientes, Punto de Venta, Ventas, Logística/Inventario y Reportes, la práctica totalidad

aprobó sin observaciones, validando reglas de negocio como el bloqueo de campos obligatorios vacíos, el rechazo de montos de apertura de caja negativos o fuera de rango, el bloqueo de decremento de cantidad en el carrito por debajo de 1, el rechazo de sobrepago en pagos parciales y la restricción de acceso al módulo de Reportes para el rol Cajero. El bloque de Dashboard y el bloque de Reportes concentraron los defectos detectados en este nivel, resumidos en la Tabla V.

Tabla V
DEFECTOS DETECTADOS EN PRUEBAS FUNCIONALES MANUALES.

ID	Descripción
ID-008	Inconsistencia en el refresco y cálculo de movimientos de una caja abierta en el Dashboard.
ID-009	El saldo y los movimientos de una caja abierta sin transacciones no se reflejan según lo esperado.
DEF-SU-01	El filtro de “productos de mayor rotación” no está implementado en la interfaz de Reportes.
DEF-SU-02	El indicador de Ticket Promedio no está desarrollado ni se muestra en la aplicación.

Los cuatro defectos de este nivel corresponden a funcionalidad de reporting no crítica para la operación diaria de venta (Dashboard de caja e indicadores de Reportes), a diferencia de los defectos de permisos detectados en el nivel unitario, que sí comprometían el control de acceso. Este contraste ilustra la complementariedad de niveles: la caja blanca detecta defectos de lógica interna antes de exponer la interfaz, y la caja negra detecta huecos de alcance -funcionalidad especificada pero no implementada- que solo son visibles al operar el sistema como lo haría un usuario final.

VI. PRUEBAS DE INTEGRACIÓN

Las pruebas de integración verifican la interacción entre módulos del backend contra una base de datos PostgreSQL real, a diferencia de las pruebas unitarias que aíslan cada componente mediante mocks. Se diseñaron e implementaron 10 escenarios sobre el flujo crítico Ventas–Productos–Caja–Secuencias–Servicios Externos, resumidos en la Tabla VI. Los diez casos aprobaron en su ejecución más reciente, cubriendo el descuento de stock tras una venta, el rollback ante stock insuficiente, la actualización del saldo de caja, el rechazo de ventas sin sesión de caja abierta, la emisión de tokens JWT, la denegación de acceso sin cabecera *Bearer*, la respuesta ante parámetros inválidos en los proxies de DNI y RUC, y el rechazo de operaciones administrativas a un usuario de nivel inferior (vendedor frente a gerente).

VII. PRUEBAS DE SISTEMA E2E

VII-A. Catálogo

El catálogo de sistema de extremo a extremo se diseñó con Playwright sobre cuatro rutas críticas de negocio: Punto de Venta y Caja (7 escenarios), Catálogo e Inventario (4), Autenticación y Control de Acceso (4), y Gestión de Clientes y Recetas Clínicas (2), totalizando 17 escenarios de negocio. Varios de estos escenarios se materializan en más de un

Tabla VI
CATÁLOGO DE PRUEBAS DE INTEGRACIÓN (INT-01 A INT-10).

ID	Caso	Resultado
INT-01	Venta descuento stock del producto	Aprobado
INT-02	Rollback ante stock insuficiente	Aprobado
INT-03	Venta actualiza saldo de caja abierta	Aprobado
INT-04	Venta sin caja activa es rechazada	Aprobado
INT-05	Adquisición y uso de tokens JWT	Aprobado
INT-06	Proxy DNI requiere autenticación	Aprobado
INT-07	Proxy DNI ante parámetro inválido	Aprobado
INT-08	Permisos por nivel (vendedor vs. gerente)	Aprobado
INT-09	Proxy RUC requiere autenticación	Aprobado
INT-10	Proxy RUC ante parámetro inválido	Aprobado

caso de prueba automatizado cuando cubren tanto el camino normal como una frontera AVL -por ejemplo, E2E-INV-03 valida en un único escenario de negocio tres casos concretos (agregar exactamente 1 unidad, bloquear la unidad excedente y bloquear el decremento por debajo de 1)-, lo que produce 24 funciones de prueba (`test()`) concretas en el código. Estas 24 pruebas se ejecutan sobre los tres motores soportados (Chromium, Firefox y WebKit) mediante la matriz de proyectos de Playwright, produciendo 72 ejecuciones automatizadas por corrida completa de la suite, verificadas el 19/07/2026 mediante `playwright test-list`.

VII-B. Resultados

La ejecución documentada de los 17 escenarios de negocio en el Informe de Pruebas de Sistema E2E registra resultado satisfactorio en la totalidad de los casos, incluyendo la apertura de caja con AVL sobre el monto inicial, la venta de una montura con stock unitario, la venta con luna personalizada de precio manual, el pago parcial y el cobro de saldo pendiente, el cierre de caja con arqueo, la anulación de venta con devolución de stock, el alta de producto con stock en la frontera cero, la búsqueda y filtrado combinado de inventario, la gestión de proveedores con validación de RUC, el bloqueo de rutas por nivel de acceso en los cuatro roles, la persistencia de sesión tras recarga, y el registro de cliente con DNI duplicado y de receta óptica. La Tabla VII resume el catálogo por ruta crítica.

Tabla VII
CATÁLOGO DE PRUEBAS DE SISTEMA E2E POR RUTA CRÍTICA.

Ruta crítica	Escenarios	Resultado
POS y Gestión de Caja	7	7/7 satisfactorio
Catálogo e Inventario	4	4/4 satisfactorio
Autenticación y Control de Acceso	4	4/4 satisfactorio
Cientes y Recetas Clínicas	2	2/2 satisfactorio
Total	17	17/17

Una ejecución intermedia de compatibilidad entre navegadores (Sección VIII, NF-COMPAT-01) sí detectó, en una corrida previa a la consolidación final, una falla reproducible de forma idéntica en Chromium y en Firefox sobre el escenario de bloqueo de rutas para el rol de nivel 2, atribuida a desincronización entre los selectores de Playwright y el DOM real de

la aplicación y no a un problema de interpretación de motor de navegador. Este hallazgo, junto con otros ajustes de estabilidad de la suite (reemplazo de esperas fijas `waitForTimeout` por esperas dinámicas, y corrección de los datos semilla de base de datos para evitar errores de integridad en ejecuciones consecutivas), se resolvió en un commit de estabilización de la canalización de CI/CD posterior (eb91b69, “fix: resolve CI e2e pipeline failures in qa”), validado localmente con la suite completa de 3 navegadores: 72 pruebas ejecutadas, 0 fallidas.

VIII. PRUEBAS NO FUNCIONALES

Las pruebas no funcionales se organizaron en tres categorías -estrés y carga, seguridad, y calidad extendida conforme a ISO/IEC 25010- totalizando 17 escenarios, de los cuales 11 aprobaron y 6 no aprobaron los criterios del plan, según se resume en la Tabla VIII.

Tabla VIII
RESUMEN DE RESULTADOS NO FUNCIONALES POR CATEGORÍA.

Categoría	Escenarios	Aprobados	No aprobados
Estrés y carga (Locust)	6	2	4
Seguridad (pytest)	7	7	0
Calidad extendida (ISO 25010)	4	2	2
Total	17	11	6

VIII-A. Estrés y carga

Se implementaron seis escenarios con Locust [10] sobre los endpoints más críticos, con estrategias de estrés realista: pool rotativo de productos, payloads variados, formas de pago distribuidas aleatoriamente y auto-recuperación ante expiración de token. La Tabla IX resume las métricas obtenidas.

Tabla IX
RESULTADOS DE PRUEBAS DE ESTRÉS Y CARGA.

ID	Usuarios	P95	Error	Estado
NF-STRESS-01 (ventas)	50	720 ms	0 %	Aprobado
NF-STRESS-02 (búsqueda)	100	46 s	0 %	No aprobado
NF-STRESS-03 (carrera)	10	3.8 s	0 %	Aprobado
NF-STRESS-04 (login)	100	7.4 s	16.7 %	No aprobado
NF-STRESS-05 (dashboard)	20	11 s	0 %	No aprobado
NF-SPIKE-01 (pico 0-200)	200	91 s	0 %	No aprobado

El escenario de mayor relevancia para la integridad de datos, NF-STRESS-03, sometió a 10 usuarios concurrentes a comprar el mismo producto con stock = 1: Locust registró exactamente 1 venta exitosa (HTTP 201) y 9 rechazos por stock insuficiente (HTTP 400), confirmado además por una consulta SQL directa post-prueba que verificó stock final = 0 y una única venta registrada, sin condición de carrera real. Los cuatro escenarios no aprobados (NF-STRESS-02, 04, 05 y NF-SPIKE-01) no comprometieron la integridad de los datos -el error rate por fallas del sistema fue 0 % en tres de los cuatro, y el sistema permaneció disponible incluso en el pico de 200 usuarios- pero sí excedieron ampliamente los umbrales de latencia P95 definidos en el plan (2-8 s según el escenario), evidenciando

cuellos de botella concretos: el endpoint de búsqueda de productos (GET /api/products/), el endpoint de estadísticas del dashboard, y el endpoint de emisión de tokens JWT bajo un pico simultáneo de 100 solicitudes.

VIII-B. Seguridad

Se implementaron siete escenarios con `pytest` y el `APIClient` de DRF, cubriendo autenticación, autorización por rol, expiración de JWT e inyección SQL, alineados con las categorías de OWASP API Security Top 10 [11]. Los siete escenarios aprobaron sin excepción, resumidos en la Tabla X.

Tabla X
RESULTADOS DE PRUEBAS DE SEGURIDAD.

ID	Verificación	Resultado
NFSEC-01	GET sin token JWT → 401	Aprobado
NFSEC-02	POST/PUT/PATCH/DELETE sin token → 401	Aprobado
NFSEC-03	Vendedor accede a listado de usuarios → 403	Aprobado
NFSEC-04	Vendedor accede a listado de cajeros → 403	Aprobado
NFSEC-05	Token JWT expirado → 401	Aprobado
NFSEC-06	Inyección SQL en búsqueda de productos	Aprobado
NFSEC-07	Inyección SQL en búsqueda de ventas	Aprobado

Los payloads de inyección (DROP TABLE, UNION SELECT, OR 1=1, entre otros) enviados a los endpoints de búsqueda de productos y ventas produjeron únicamente respuestas HTTP 200 o 400, sin errores internos (HTTP 500) ni evidencia de ejecución de SQL arbitrario, lo que confirma que el uso sistemático del ORM de Django mitiga este vector de ataque sin necesidad de sanitización manual adicional.

VIII-C. Calidad extendida (ISO/IEC 25010)

Se ejecutaron cuatro escenarios adicionales orientados a subcaracterísticas de calidad no cubiertas por las categorías anteriores. NF-VOL-01 evaluó el comportamiento ante un catálogo de 100 000 productos (frente a una línea base de ~1000): el percentil 95 de búsqueda alcanzó 33 s en la suite completa de consultas (frente a 8.4 s en la línea base de solo búsqueda por marca), superando el umbral de 3 s del plan y evidenciando la necesidad de optimizar índices y el planificador de consultas para catálogos de ese orden de magnitud. NF-SOAK-01 sometió al sistema a 30 minutos de carga sostenida con 5 usuarios ejecutando un ciclo realista de login, búsqueda, venta y consulta de dashboard: la latencia P95 *disminuyó* de 1 128 ms en la primera ventana de 5 minutos a 589 ms en la última, sin fallos registrados, lo que descarta degradación progresiva o fuga de recursos perceptible en ese horizonte de tiempo. NF-REC-01, automatizado como job del pipeline de CI/CD, interrumpió abruptamente el contenedor de PostgreSQL durante 30 segundos bajo carga concurrente de 10 usuarios: tras el reinicio, el login volvió a responder HTTP 200, y la verificación SQL posterior confirmó cero productos con stock negativo y cero ventas en estados inconsistentes, validando que el motor transaccional de PostgreSQL

preserva la integridad ante una caída de infraestructura. NF-COMPAT-01 ejecutó la suite completa de 7 escenarios E2E en Chromium y Firefox -WebKit no pudo ejecutarse en el entorno local por falta de dependencias nativas-, con un fallo idéntico y reproducible en ambos motores, atribuido a un defecto de la aplicación (posteriormente corregido, Sección VII) y no a incompatibilidad entre navegadores.

IX. PRUEBAS DE ACEPTACIÓN (UAT)

IX-A. Diseño de la sesión

La sesión de aceptación se ejecutó como prueba Beta formal [6] el 8 de julio de 2026, con una usuaria externa real ajena al equipo de desarrollo, sobre un despliegue accesible remotamente (VPS/Ngrok) desde el propio entorno de la validadora (Windows 11, navegador Brave), sin instalación local. La sesión, de 70 minutos de duración frente a los 60 planificados, se grabó en video y se registró en tiempo real sobre una hoja de cálculo colaborativa, cubriendo 12 de los 15 criterios documentados en el plan, agrupados en Autenticación, POS y Caja, Pagos y Anulaciones, Inventario, Reportes y Configuración, y consulta de DNI vía la API externa.

IX-B. Resultados

De los 12 escenarios ejecutados, 11 fueron validados por la usuaria (91.6 %), por encima del umbral mínimo de aceptación del 83.3 % (10 de 12) definido en el plan, y la totalidad de los criterios obligatorios (login, apertura de caja, venta, cierre de caja, inventario y bloqueo de accesos no autorizados) se cumplió. El único criterio no validado fue UAT-09 (gestión de usuarios): el sistema permite iniciar la creación de un usuario con un correo ya registrado sin advertir la colisión hasta después de intentar guardar, en lugar de validar la unicidad de forma anticipada en el formulario. Este hallazgo se registró como incidente INC-001 de severidad media. El veredicto formal de la sesión fue aprobado condicionalmente, con reservas.

Además del defecto formal, la usuaria aportó seis observaciones de mejora de usabilidad recogidas durante el uso prolongado y no anticipadas en las pruebas técnicas previas: exigir que el monto inicial de apertura de caja se ingrese explícitamente en lugar de aceptar un valor por defecto; permitir convertir un cliente genérico en cliente formal inmediatamente después de una venta; habilitar texto libre en la opción “Otro” de material/marca de lunas personalizadas; corregir que la barra de búsqueda de inventario se deselectione al escribir y que el resultado se pierda al ajustar stock; y mostrar el código de producto en el detalle de venta para verificar con mayor rapidez las devoluciones de stock tras una anulación. Estas observaciones, si bien no bloquean la aprobación condicionada, quedan documentadas como riesgos residuales para la puesta en producción.

X. MATRIZ DE TRAZABILIDAD Y COBERTURA

La Tabla XI consolida la cobertura de los diez módulos del backend a través de los niveles de prueba unitario, de integración, de sistema E2E, no funcional y de

aceptación. Los cuatro módulos sin cobertura más allá del nivel unitario (`suppliers`, `categories`, `sequences`, `opticalCenter`) concentran lógica de negocio de baja complejidad -validaciones de formato, generación de correlativos, operaciones CRUD sin flujos transaccionales- lo que hace que la cobertura unitaria sea proporcional al riesgo que representan; en particular, `opticalCenter` implementa un patrón Singleton de solo lectura/configuración sin flujo de usuario que justifique una prueba E2E independiente.

En términos de las ocho características de calidad de ISO/IEC 25010, seis quedan cubiertas por al menos una prueba del catálogo: Adecuación Funcional (unitarias, funcionales y E2E), Eficiencia de Desempeño (NF-STRESS, NF-VOL), Compatibilidad (NF-COMPAT), Usabilidad (UAT-15), Fiabilidad (NF-SOAK, NF-REC) y Seguridad (NFSEC, E2E-AUTH-03). Mantenibilidad queda fuera de alcance por requerir análisis estático de código en lugar de pruebas dinámicas, y Portabilidad queda fuera de alcance por exclusión explícita del empaquetado Tauri en el plan maestro.

XI. INTEGRACIÓN CONTINUA

El proyecto mantiene dos flujos de trabajo de GitHub Actions activos sobre las ramas `develop` y `qa`. El flujo de `qa` ejecuta, en cada *push* o *pull request* contra esa rama, un job de pruebas unitarias e integración con cobertura, un job de seguridad, un job `e2e` que levanta PostgreSQL, aplica migraciones y datos semilla, inicia el backend y el frontend, y ejecuta la suite completa de Playwright en modo headless, y un job `recoverability` que automatiza el escenario NF-REC-01 descrito en la Sección VIII. Solo si estos jobs aprueban se autoriza la construcción de la imagen Docker y su despliegue por SSH al servidor de QA. Como parte del mantenimiento continuo de esta infraestructura, se identificó y eliminó, durante el cierre del proyecto, un flujo de trabajo de Playwright duplicado y huérfano en la raíz del repositorio -remanente de un andamiaje generado por error fuera del directorio `frontend/`- que apuntaba a un directorio de pruebas inexistente y que, de haberse activado, habría producido sistemáticamente un falso negativo de “*No tests found*” independiente del estado real de la suite; su eliminación se verificó sin afectar la colección de las 72 ejecuciones E2E ni de las 518 pruebas de backend.

XII. DISCUSIÓN

El proceso de verificación y validación aplicado a RegistraMe permite distinguir con claridad dos dimensiones de calidad con resultados opuestos. En corrección funcional, control de acceso e integridad transaccional, el sistema es sólido: la totalidad de los escenarios de seguridad (7/7), la totalidad de los escenarios de sistema E2E tras su estabilización (17/17), la totalidad de las pruebas de integración (10/10) y el escenario de condición de carrera sobre stock crítico aprobaron sin reservas, y la prueba de recuperabilidad confirmó que el motor transaccional de PostgreSQL preserva la consistencia de los datos incluso ante una caída abrupta de la base de datos bajo carga. En eficiencia de desempeño bajo concurrencia,

en cambio, el sistema presenta cuellos de botella concretos y reproducibles: la búsqueda de productos, la agregación de estadísticas del dashboard y la emisión masiva de tokens JWT degradan su latencia muy por encima de los umbrales definidos (RNF-01 exige un máximo de 3 s) tan pronto la concurrencia supera el orden de las decenas de usuarios simultáneos, un patrón consistente entre NF-STRESS-02, NF-STRESS-05, NF-STRESS-04 y NF-VOL-01 que apunta a un origen común probable en la ausencia de índices adecuados y de paginación eficiente sobre las consultas de agregación, más que a fallos aislados por endpoint.

La sesión de aceptación aporta una tercera dimensión que ninguna prueba técnica automatizada puede sustituir: la ausencia de validación anticipada de correos duplicados en la creación de usuarios (UAT-09) no había sido detectada por la suite unitaria ni por la suite E2E, porque ambas ejercitan condiciones que sus propios diseñadores anticipan, mientras que el usuario final expone exactamente los caminos que el equipo no anticipó. Este hallazgo, junto con las seis observaciones de usabilidad recogidas durante la sesión, confirma el valor de incorporar una prueba Beta con usuario real como cierre del ciclo de pruebas, incluso sobre un sistema que ya cuenta con una suite automatizada extensa en los niveles inferiores.

En conjunto, los resultados sustentan una recomendación clara antes de un despliegue a producción con concurrencia real: priorizar la optimización de los tres endpoints identificados (búsqueda de productos, estadísticas de dashboard, emisión de tokens) y la implementación de validación anticipada de unicidad de correo en el alta de usuarios, manteniendo sin cambios la arquitectura de seguridad y de integridad transaccional, que ya satisface los criterios definidos.

XIII. CONCLUSIONES

Este informe documentó el proceso íntegro de verificación y validación aplicado al sistema RegistraMe a lo largo de cuatro sprints, estructurado en cinco niveles de prueba diseñados mediante Partición de Equivalencia, Análisis de Valores Límite y el Principio de la Mochila, bajo el marco documental de ISO/IEC/IEEE 29119-3 y el modelo de calidad ISO/IEC 25010. La suite automatizada asciende, al cierre del proyecto, a 518 pruebas unitarias de backend y 72 ejecuciones de sistema E2E sobre tres motores de navegador, complementadas por 10 pruebas de integración, 17 escenarios no funcionales y una sesión formal de aceptación con usuario externo real. El sistema demuestra corrección funcional, control de acceso robusto e integridad transaccional ante condiciones de carrera y fallos de infraestructura, y presenta oportunidades de optimización de rendimiento concretas y acotadas -búsqueda de inventario, dashboard y autenticación masiva- junto con un defecto de validación de datos identificado directamente por el usuario final, que en conjunto constituyen el backlog de calidad priorizado para la siguiente iteración del proyecto antes de su despliegue a producción.

Tabla XI
MATRIZ DE TRAZABILIDAD MÓDULO – NIVEL DE PRUEBA.

Módulo	Integración	E2E	No funcionales	UAT
users	INT-05, INT-08	AUTH-01 a 04	STRESS-04, SEC-01 a 04	01, 09, 12
sales	INT-01 a 04	POS-02 a 07	STRESS-01, STRESS-03	03 a 06, 11
products	INT-01, INT-02	INV-01 a 03	STRESS-02, VOL-01	08
cash	INT-03, INT-04	POS-01, POS-06	STRESS-05	02, 06
clients	—	CLI-01	—	10
suppliers	—	INV-04	—	—
external_services	INT-06, 07, 09, 10	(implicito)	—	10

REFERENCIAS

- [1] ISO/IEC/IEEE, “ISO/IEC/IEEE 29119-3:2021 — software testing — part 3: Test documentation,” International Organization for Standardization, Tech. Rep., 2021.
- [2] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Wiley, 2011.
- [3] ISO/IEC/IEEE, “ISO/IEC/IEEE 29119-4:2021 — software testing — part 4: Test techniques,” International Organization for Standardization, Tech. Rep., 2021.
- [4] A. Spillner, T. Linz, and H. Schaefer, *Software Testing Foundations*, 4th ed. dpunkt.verlag, 2014.
- [5] ISO/IEC, “ISO/IEC 25010:2023 — systems and software quality models,” International Organization for Standardization, Tech. Rep., 2023.
- [6] R. S. Pressman, *Software Engineering: A Practitioner’s Approach*, 7th ed. McGraw-Hill, 2010.
- [7] pytest-django Contributors, “pytest-django: A django plugin for pytest,” 2026, <https://pytest-django.readthedocs.io>.
- [8] Encode OSS Ltd., “Django REST framework,” 2026, <https://www.django-rest-framework.org>.
- [9] Microsoft Corporation, “Playwright: Fast and reliable end-to-end testing for modern web apps,” 2026, <https://playwright.dev>.
- [10] Locust.io Contributors, “Locust: An open source load testing tool,” 2026, <https://locust.io>.
- [11] OWASP Foundation, “OWASP API security top 10 — 2023,” OWASP Foundation, Tech. Rep., 2023.